UTM online

Universidad Técnica de Manabí desde 1952

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: JEAN FRANCISCO LESCANO MEJÍA

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: 1

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

Introducción

El presente proyecto tiene como finalidad diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal, orientado a facilitar la gestión de los animales que ingresan a un refugio.

El sistema permitirá registrar casos de rescate, clasificar animales según su especie, asignar voluntarios y veterinarios responsables, y realizar un seguimiento de su estado hasta llegar al proceso de adopción.

En esta primera etapa se realizará el análisis de los requerimientos, el diseño de diagramas UML y la selección de patrones de diseño de software. Además, se utilizará GitHub como herramienta de control de versiones, aplicando el flujo de trabajo Gitflow para organizar los avances del proyecto.

Análisis del problema

Los refugios de animales necesitan mantener un control organizado de los rescates, tratamientos veterinarios y procesos de adopción. Cuando esta información se administra manualmente o mediante registros dispersos, pueden presentarse dificultades para consultar el historial de cada animal, conocer su estado actual y determinar quién es el responsable de su atención.

Ante esta situación, se propone diseñar un Sistema de Control de Rescate y Adopción Animal que permita centralizar la información, facilitar la asignación de responsabilidades y dar seguimiento a cada caso desde el rescate hasta su posible adopción.

El sistema estará orientado a gestionar perros, gatos y fauna silvestre, considerando que esta última requiere procedimientos especiales y no necesariamente puede ser entregada en adopción.

Actores del sistema

Los actores son las personas que interactuarán con el Sistema de Control de Rescate y Adopción Animal. Se identifican los siguientes:

Ciudadano o reportante: Persona que informa sobre un animal que necesita ser rescatado, proporcionando su ubicación y una descripción de la situación.

Voluntario: Encargado de atender los rescates asignados, trasladar animales al refugio y actualizar la información del rescate.

Veterinario: Profesional responsable de evaluar la salud de los animales rescatados, registrar diagnósticos y tratamientos, y determinar si están en condiciones de continuar con el proceso de adopción.

Administrador del refugio: Responsable de gestionar los casos de rescate, asignar voluntarios y veterinarios, supervisar los estados de los animales y administrar los procesos de adopción.

Adoptante: Persona interesada en adoptar un animal disponible, que podrá consultar información y presentar una solicitud de adopción.

Requerimientos funcionales

Los requerimientos funcionales describen las acciones que podrá realizar el Sistema de Control de Rescate y Adopción Animal.

RF01. Registrar casos de rescate: Permitir que los ciudadanos reporten animales que necesitan ser rescatados, indicando su ubicación y situación.

RF02. Registrar animales: Almacenar información de cada animal rescatado, incluyendo especie, características y fecha de ingreso.

RF03. Clasificar especies: Identificar a los animales como perros, gatos o fauna silvestre.

RF04. Asignar responsables: Permitir al administrador asignar voluntarios y veterinarios a los casos de rescate.

RF05. Registrar atención veterinaria: Guardar diagnósticos, tratamientos y observaciones médicas.

RF06. Actualizar estados: Gestionar estados como Reportado, Rescatado, En tratamiento, Disponible para adopción y Adoptado.

RF07. Gestionar adopciones: Registrar solicitudes de adopción y permitir al administrador aprobarlas o rechazarlas.

RF08. Enviar notificaciones: Notificar a los voluntarios cuando se les asigne un nuevo rescate.

RF09. Consultar historial: Permitir consultar el seguimiento y los cambios registrados en cada caso de rescate.

Requerimientos no funcionales

Los requerimientos no funcionales establecen las características de calidad que deberá cumplir el Sistema de Control de Rescate y Adopción Animal.

RNF01. Escalabilidad: El sistema deberá permitir aumentar la cantidad de animales, usuarios y casos registrados sin afectar significativamente su funcionamiento.

RNF02. Mantenibilidad: El software deberá tener una estructura modular que facilite realizar correcciones, actualizaciones y mejoras.

RNF03. Seguridad: El sistema deberá proteger la información mediante autenticación y permisos según el rol de cada usuario.

RNF04. Usabilidad: La interfaz deberá ser sencilla e intuitiva para facilitar su utilización por ciudadanos, voluntarios, veterinarios y administradores.

RNF05. Disponibilidad: El sistema deberá estar disponible para registrar y consultar casos de rescate, salvo durante mantenimientos programados.

RNF06. Trazabilidad: El sistema deberá conservar un historial de cambios de los casos y utilizar GitHub para controlar las versiones del proyecto.

RNF07. Rendimiento: Las consultas y operaciones habituales deberán responder en un tiempo razonable bajo condiciones normales de uso.

Diseño UML del sistema

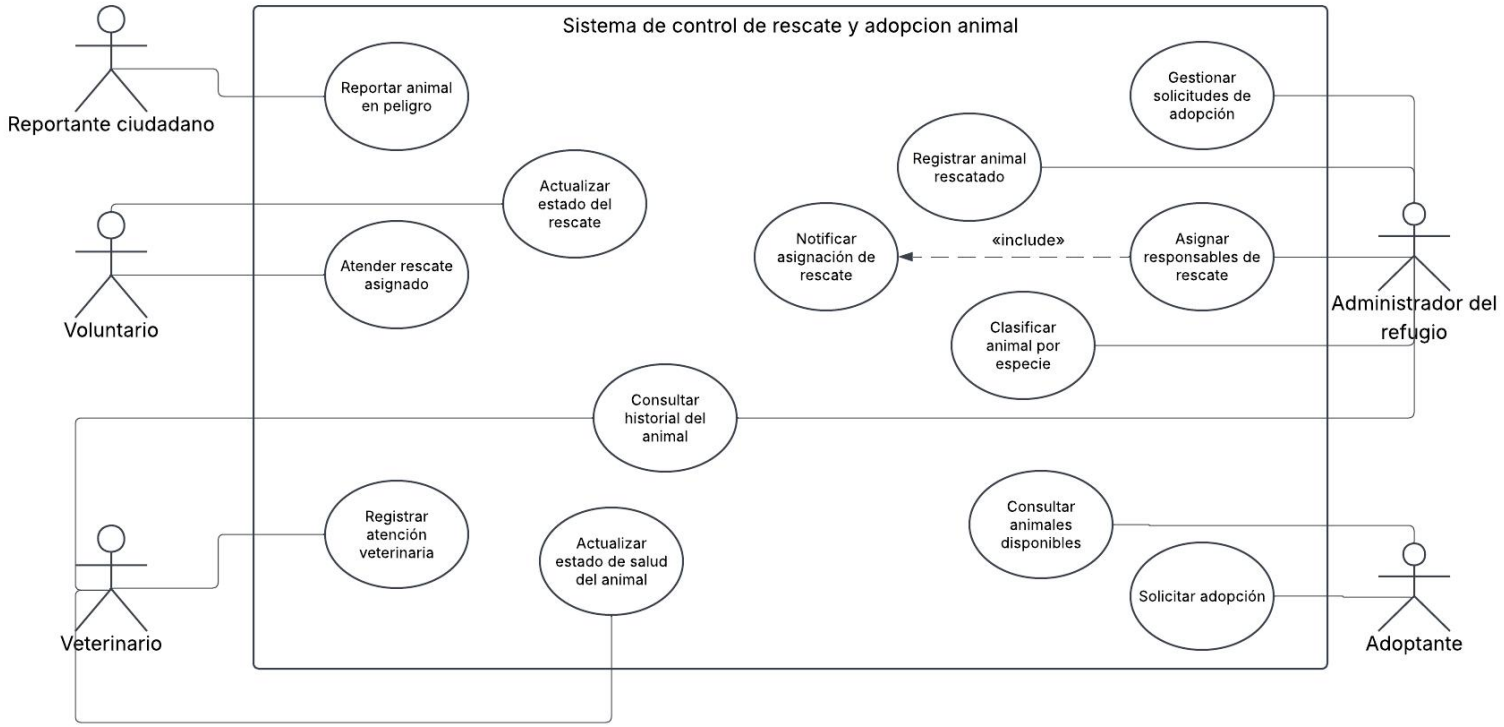


Figura 1. *Diagrama de casos de uso del sistema*

Diagrama de clases UML

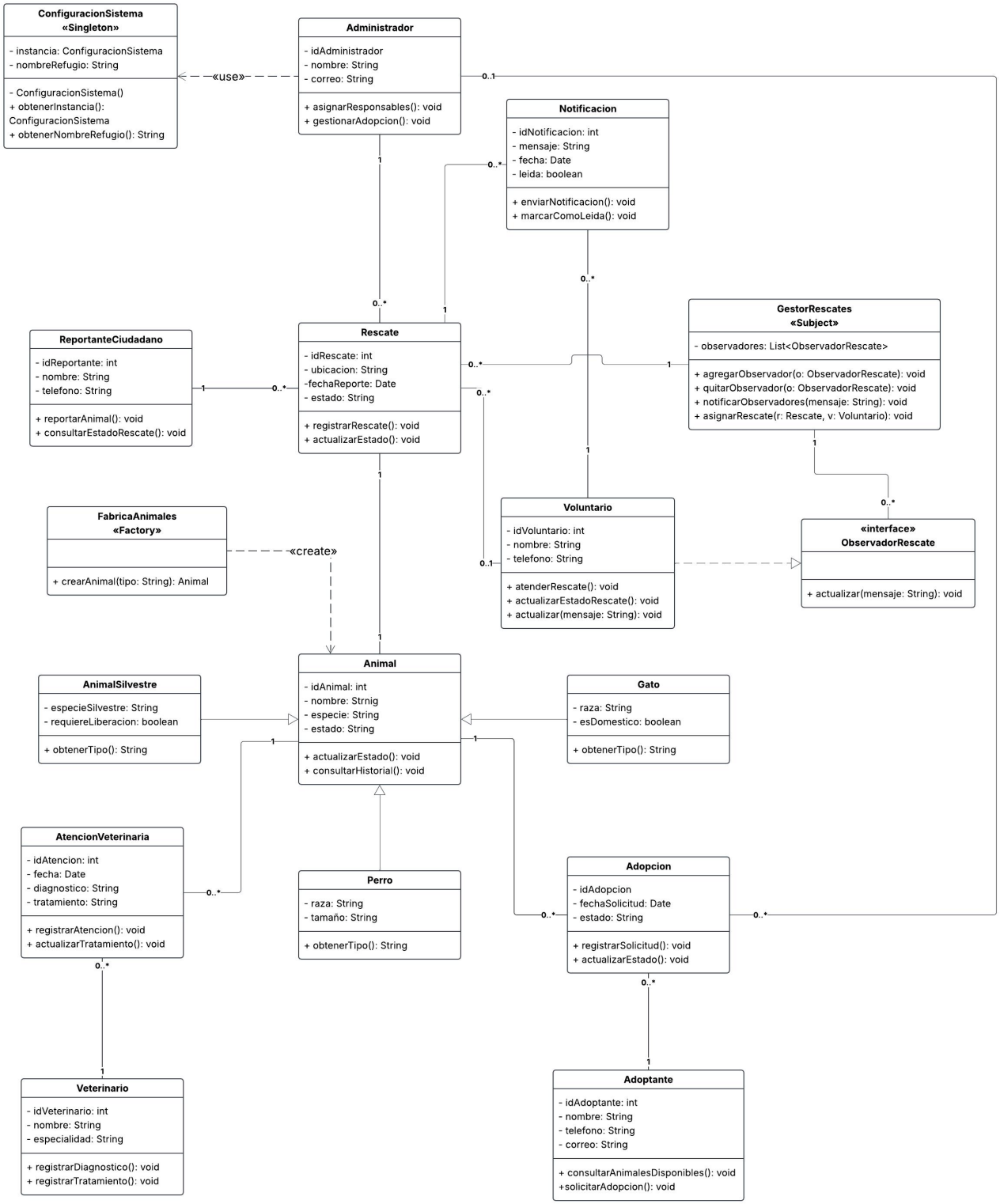


Figura 2. *Diagrama de clases UML del sistema*

Aplicación de los principios SOLID

En el diseño del sistema se consideran los principios SOLID para facilitar su mantenimiento y ampliación. El principio de responsabilidad única (SRP) se refleja en la distribución de funciones entre clases como Rescate, Animal, Adopcion y GestorRescates. El principio abierto/cerrado (OCP) se favorece mediante la herencia de la clase Animal, permitiendo incorporar nuevas especies. Asimismo, el principio de inversión de dependencias (DIP) se refleja en el uso de la interfaz ObservadorRescate, que permite gestionar las notificaciones sin depender directamente de una implementación concreta de voluntario.

Diagrama de secuencia

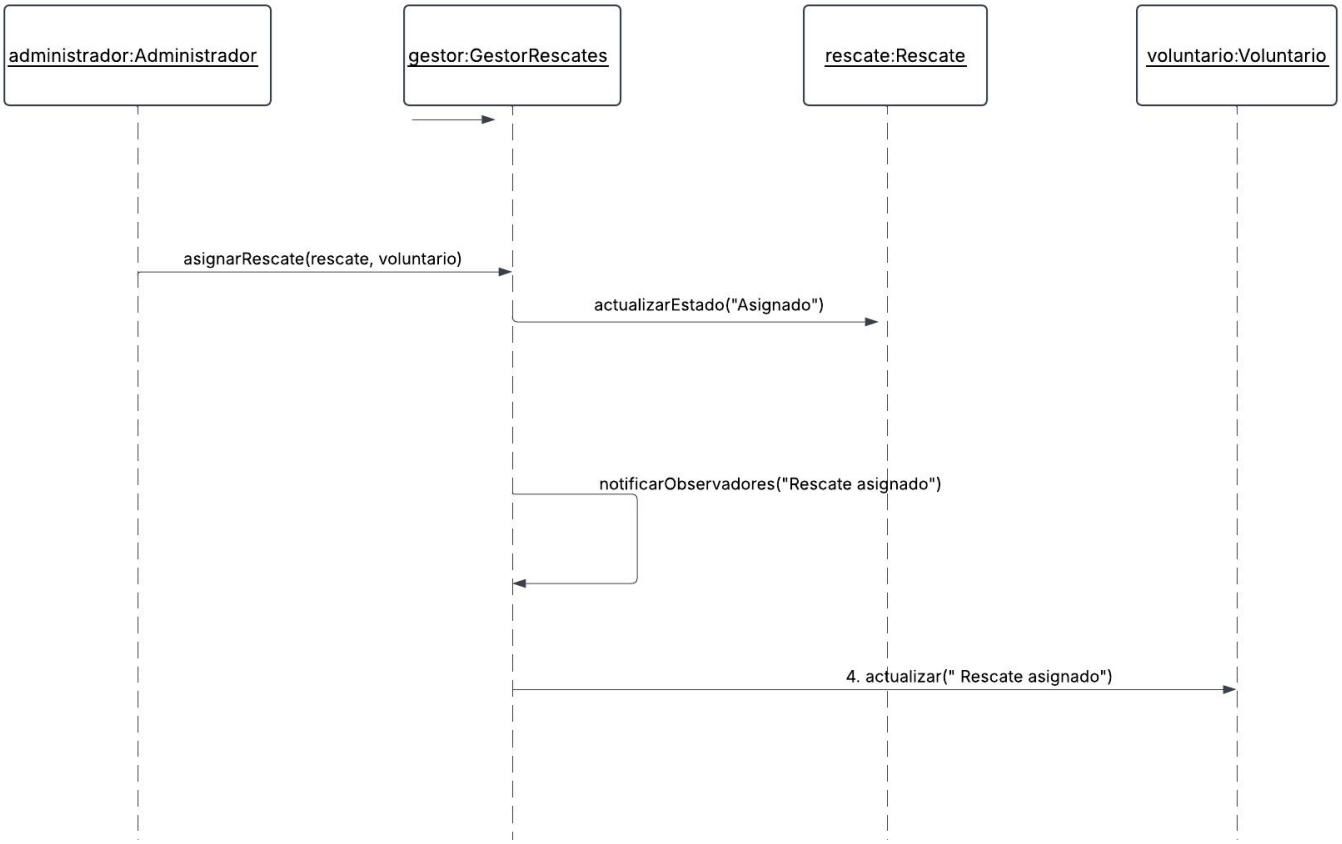


Figura 3. *Diagrama de secuencia del patrón Observer*

Arquitectura general del sistema

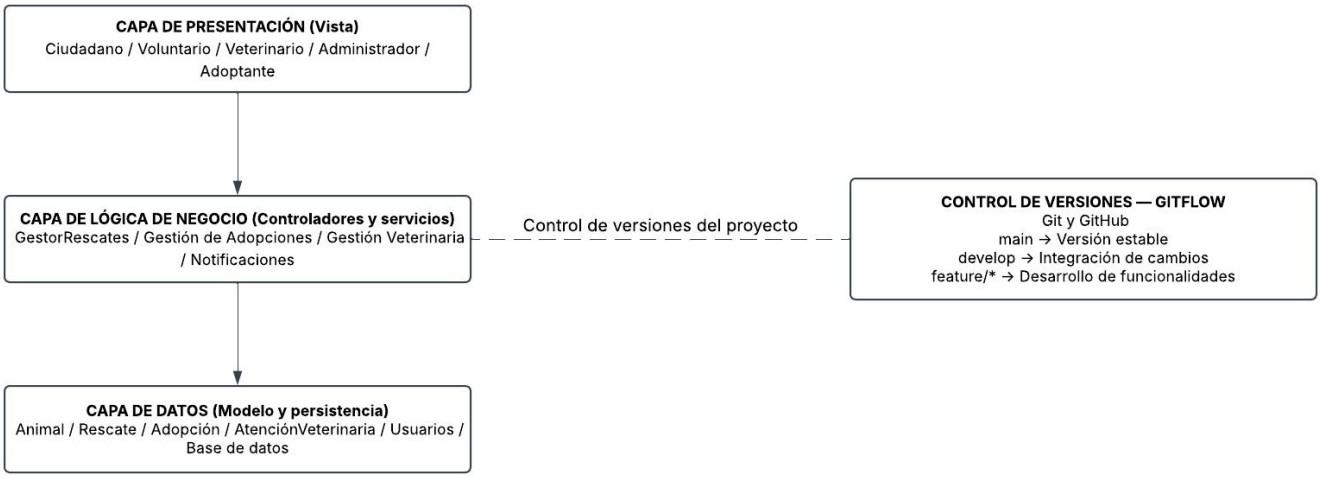


Figura 4. *Arquitectura general del sistema y Gitflow*

Patrón Singleton

El patrón Singleton se aplica en la clase ConfiguracionSistema, con el propósito de garantizar que exista una única instancia encargada de administrar la configuración general del sistema de rescate y adopción animal.

Este patrón evita la creación de múltiples objetos de configuración y permite que los diferentes componentes del sistema accedan a los mismos parámetros mediante el método obtenerInstancia().

Su aplicación favorece la consistencia de la configuración y facilita su administración centralizada.

Patrón Observer

El patrón Observer se aplica para gestionar las notificaciones relacionadas con los rescates de animales. La clase GestorRescates actúa como sujeto, mientras que los voluntarios funcionan como observadores mediante la interfaz ObservadorRescate.

Cuando se asigna un rescate o se produce un cambio relevante, el gestor ejecuta el método notificarObservadores(), que comunica la actualización a los observadores registrados mediante el método actualizar().

Este patrón permite mantener informados a los voluntarios sin generar una dependencia directa entre el gestor y las implementaciones concretas de los observadores. Además, facilita incorporar nuevos tipos de observadores en el futuro.

Patrón de fábrica (Simple Factory)

El patrón Simple Factory se aplica mediante la clase FabricaAnimales, encargada de crear objetos de diferentes especies, como Perro, Gato y AnimalSilvestre.

A través del método crearAnimal(tipo: String), el sistema selecciona la clase correspondiente y devuelve un objeto de tipo Animal, evitando que otros componentes tengan que conocer directamente cómo se crean los animales.

Este patrón centraliza la lógica de creación de objetos, facilita su mantenimiento y permite reutilizar el procedimiento de registro de animales en el sistema.

Patrón MVC (Modelo-Vista-Controlador)

El patrón MVC se utiliza para organizar la arquitectura del Sistema de Control de Rescate y Adopción Animal, separando las responsabilidades de presentación, lógica de negocio y gestión de datos.

El Modelo representa la información de los animales, rescates, usuarios y adopciones. La Vista permite que los ciudadanos, voluntarios, veterinarios y administradores interactuen con el sistema. El Controlador recibe las solicitudes de los usuarios y coordina las operaciones correspondientes.

Este patrón facilita el mantenimiento, la escalabilidad y la modificación de la interfaz sin afectar directamente la lógica del sistema.

Estrategia de control de versiones con Gitflow

Para organizar el desarrollo del Sistema de Rescate y Adopción Animal se utiliza Git como herramienta de control de versiones y GitHub como repositorio remoto.

Se aplica una estrategia basada en Gitflow mediante las siguientes ramas:

main: conserva la versión estable del proyecto.

develop: integra los avances realizados durante el desarrollo.

feature/*: permite trabajar en funcionalidades específicas antes de incorporarlas a la rama develop.

Esta organización facilita el trabajo colaborativo, permite mantener un historial de cambios mediante commits y reduce el riesgo de modificar directamente la versión estable del sistema.

El repositorio público del proyecto se encuentra disponible en:

https://github.com/JeanLescano7908/SistemaRescateAnimal

Conclusiones

El desarrollo de esta actividad permitió diseñar la arquitectura lógica de un Sistema de Rescate y Adopción Animal mediante diagramas UML, identificando los actores, los requisitos y las principales clases del sistema.

La aplicación de patrones de diseño como Singleton, Observer y Simple Factory contribuye a organizar las responsabilidades de los componentes y facilita el mantenimiento y la ampliación del sistema.

Asimismo, el uso de Git y GitHub, junto con una estrategia basada en Gitflow, permite gestionar las versiones del proyecto y mantener un registro de los cambios realizados.

En conclusión, el diseño propuesto establece una base para el futuro desarrollo e implementación del sistema, sin necesidad de programarlo en esta primera actividad.

Evidencias

Evidencias principales del control de versiones

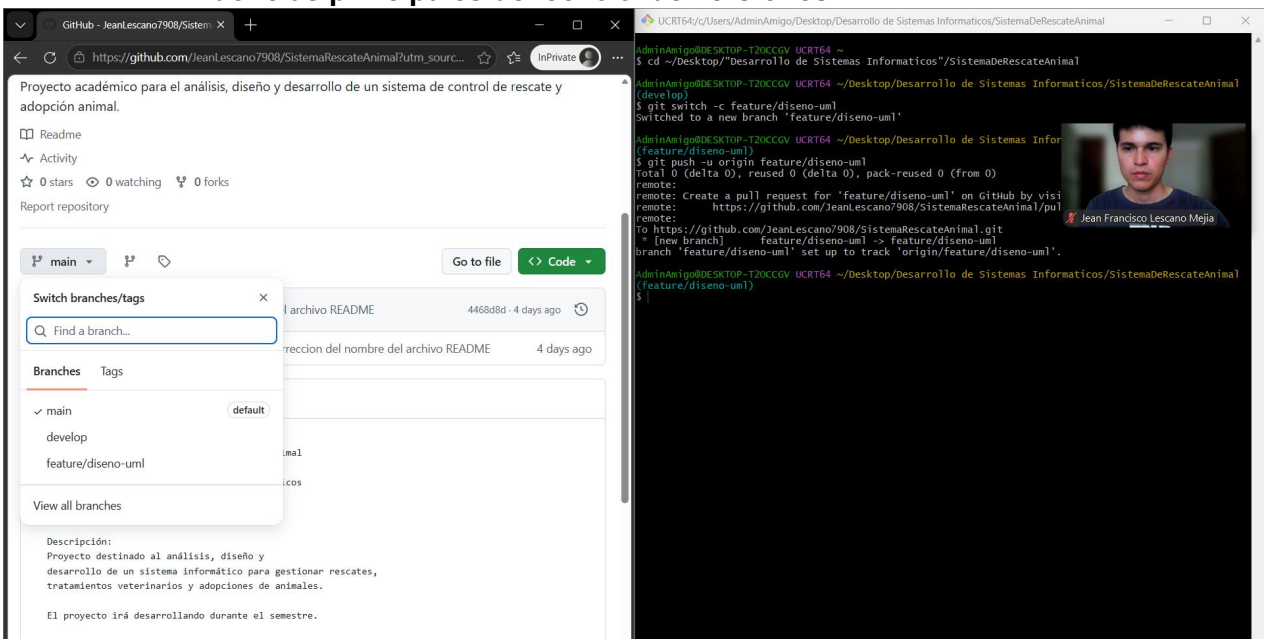


Figura 5. *Estructura de ramas del repositorio en GitHub*

La captura muestra las ramas main, develop y feature/diseno-uml del repositorio público en GitHub, evidenciando la organización del proyecto mediante la estrategia Gitflow para el control de versiones.

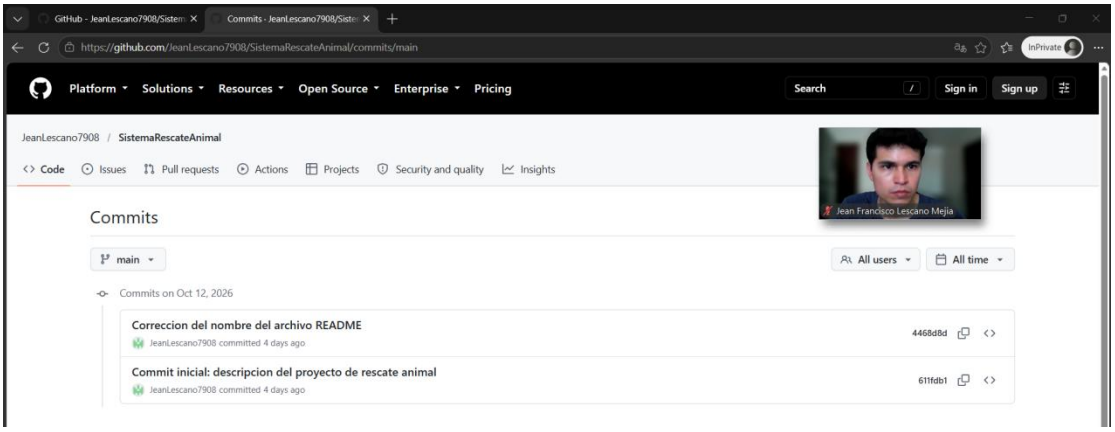
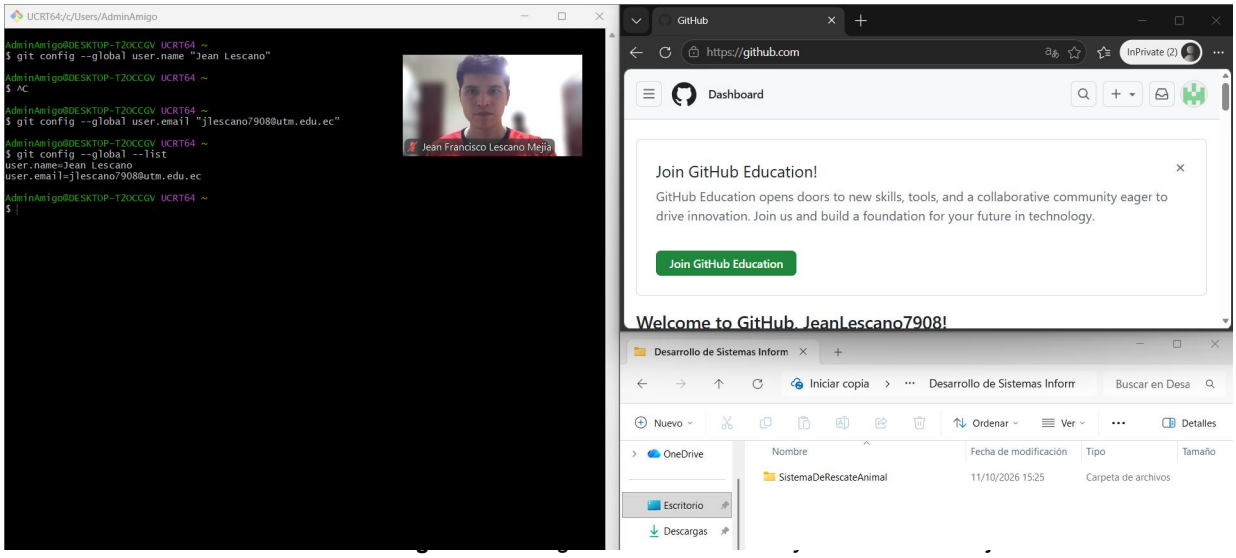


Figura 6. *Historial de commits del proyecto*

La captura presenta el historial de commits de la rama main, donde se evidencia el registro inicial del proyecto y una corrección posterior del archivo README, demostrando el uso de Git para documentar los cambios realizados.

Evidencias complementarias de configuración y publicación



La captura evidencia la configuración inicial de Git mediante Git Bash, la identificación del usuario y la preparación de la carpeta local destinada al Sistema de Control de Rescate y Adopción Animal.

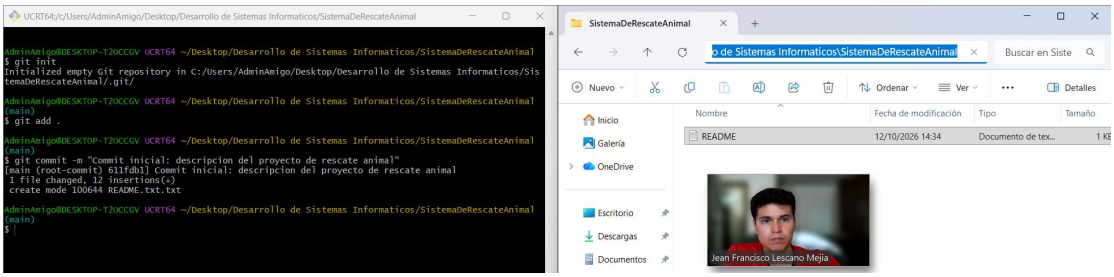


Figura 8. Inicialización del repositorio y primer commit

La captura evidencia la ejecución de comandos de Git para inicializar el repositorio local, preparar los archivos del proyecto y registrar el primer commit, estableciendo el historial inicial de control de versiones.

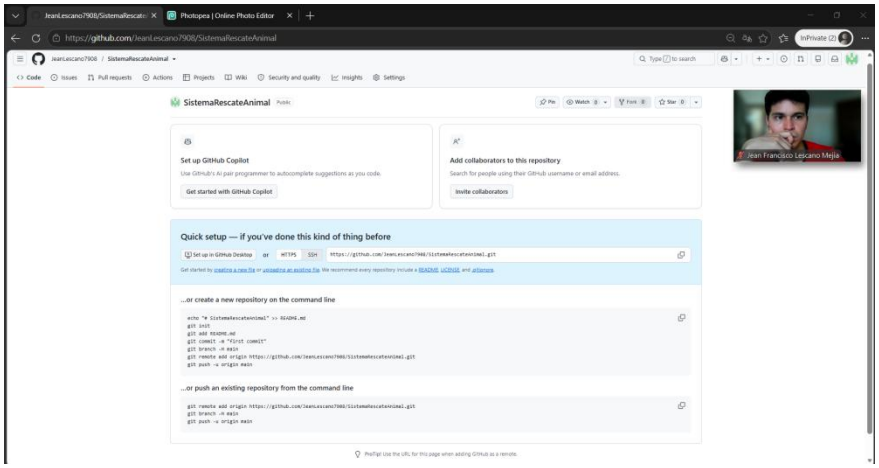


Figura 9. *Repositorio del proyecto creado en GitHub*

La captura muestra el repositorio SistemaRescateAnimal creado en GitHub, configurado para almacenar y gestionar los archivos del proyecto mediante control de versiones.

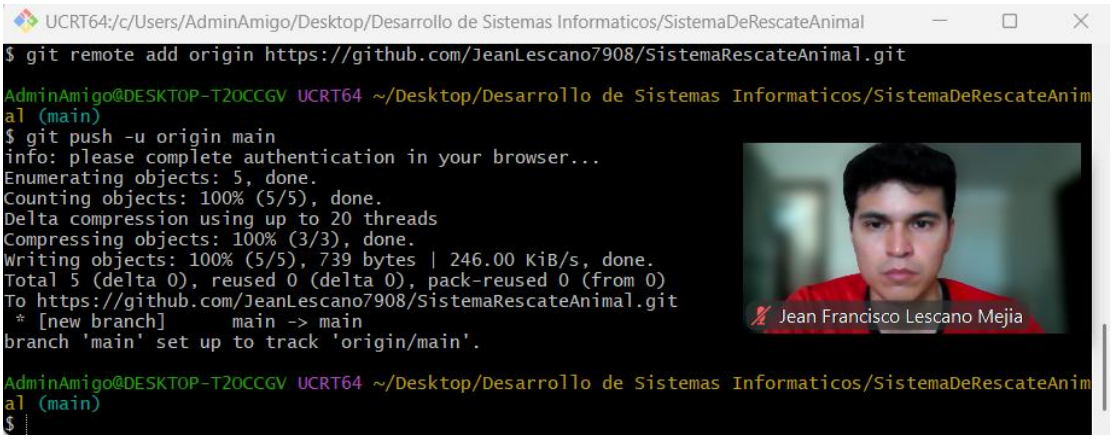


Figura 10. *Publicación inicial mediante Git Bash*

La captura evidencia la ejecución del comando git push -u origin main, utilizado para publicar los archivos y commits del repositorio local en la rama principal de GitHub.

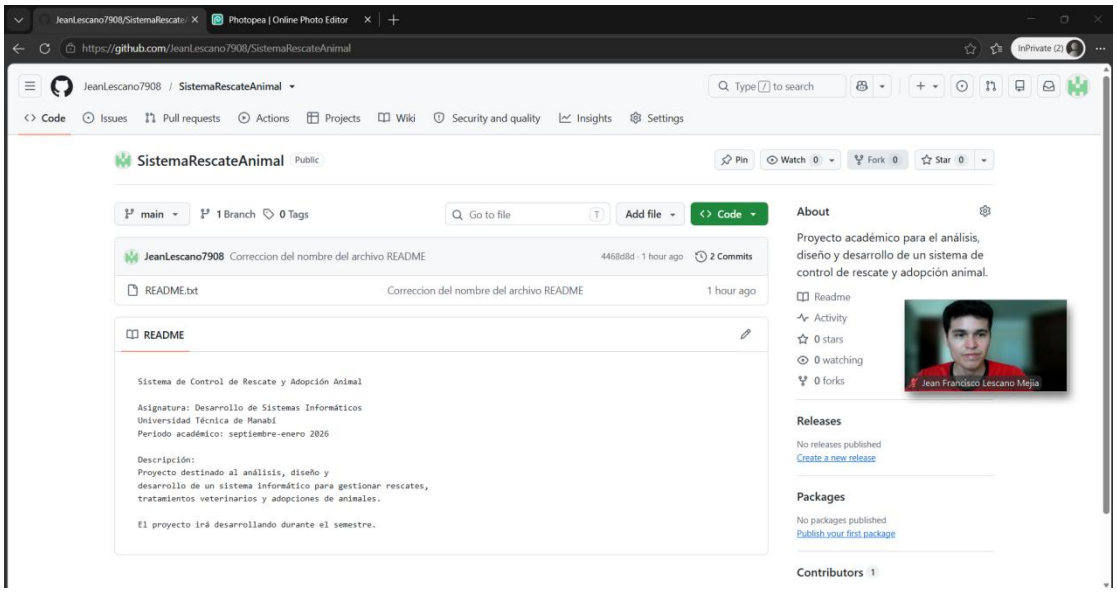


Figura 11. *Archivos del proyecto publicados en GitHub*

La captura muestra el repositorio público con el archivo README y la descripción inicial del sistema, confirmando que los archivos del proyecto fueron publicados correctamente.

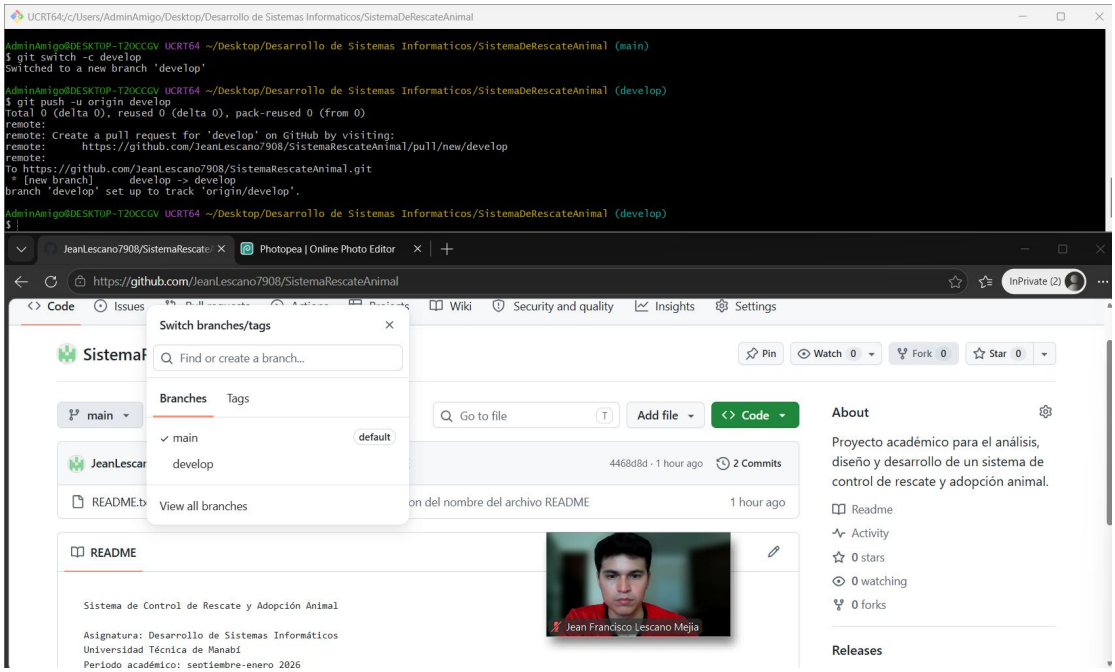


Figura 12. *Creación y publicación de la rama develop*

La captura evidencia la creación de la rama develop mediante Git Bash y su disponibilidad en GitHub, estableciendo una rama destinada a integrar los avances del proyecto sin modificar directamente la rama principal.

Enlace de los diagramas en Lucidchart:

https://lucid.app/lucidchart/f1ee3d7b-4402-4558-bfce-bb311cb8afef/edit?viewport_loc=-28%2C-1489%2C1518%2C2082%2C0_0&invitationId=inv_38ca7a56-03d6-40f2-bb96-d6542215b492

Bibliografía

Universidad Técnica de Manabí. (2025–2026). Desarrollo de Sistemas Informáticos [Compendio de la asignatura, modalidad en línea, quinto semestre, período septiembre 2025–enero 2026].

Zambrano Cedeño, J. A. (2026). Contenidos impartidos en clases de Desarrollo de Sistemas Informáticos: Análisis de sistemas, diagramas UML, patrones de diseño y control de versiones con Gitflow [Apuntes de clase]. Universidad Técnica de Manabí.

Zambrano Cedeño, J. A. (2026). Instructivo convención: Tutorial de Git y GitHub. Guía paso a paso con solución de problemas frecuentes [Material complementario en PDF disponible en el entorno virtual de aprendizaje]. Universidad Técnica de Manabí.